Experiment Name: Servo Motor

Apparatus / Requirements: Arduino UNO, Servo Motor and connecting wires

Software Used: Tinkercad **Theory**

A servo motor is a rotary actuator that can be commanded to move to a specific angle position (commonly 0° to 180°). Unlike a DC motor, a servo does not rotate freely; it holds its position based on the control signal.

Working Principle: A servo motor has 3 wires:

1. VCC (Red) → +5V
2. GND (Black/Brown) → Ground
3. Signal (Yellow/Orange) → Arduino digital pin (here D3)

The servo position is controlled using a PWM control pulse: The Arduino sends periodic control pulses.The pulse width determines the target angle. The servo's internal control circuit compares the target with its current position and rotates until it reaches the required angle, then holds that position.

In the code: servo.attach(3); connects the servo signal to digital pin 3.

servo.write(angle); sets the servo shaft to the specified angle.

Code:

```
// C++ code
#include <Servo.h>
Servo servo;  //servo is object
void setup(){
  servo.attach(3);
}
void loop(){
  for (int i=0; i<180; i++){
    servo.write(i);
    delay(10);
  }
  for (int j=179; j>=0; j--){
    servo.write(j);
  }
}
```

Result

The servo motor sweeps smoothly from 0° to ~180°, then returns back from ~179° to 0°, continuously.

This is achieved using two loops:

- First for loop increases angle step-by-step.
- Second for loop decreases angle step-by-step.

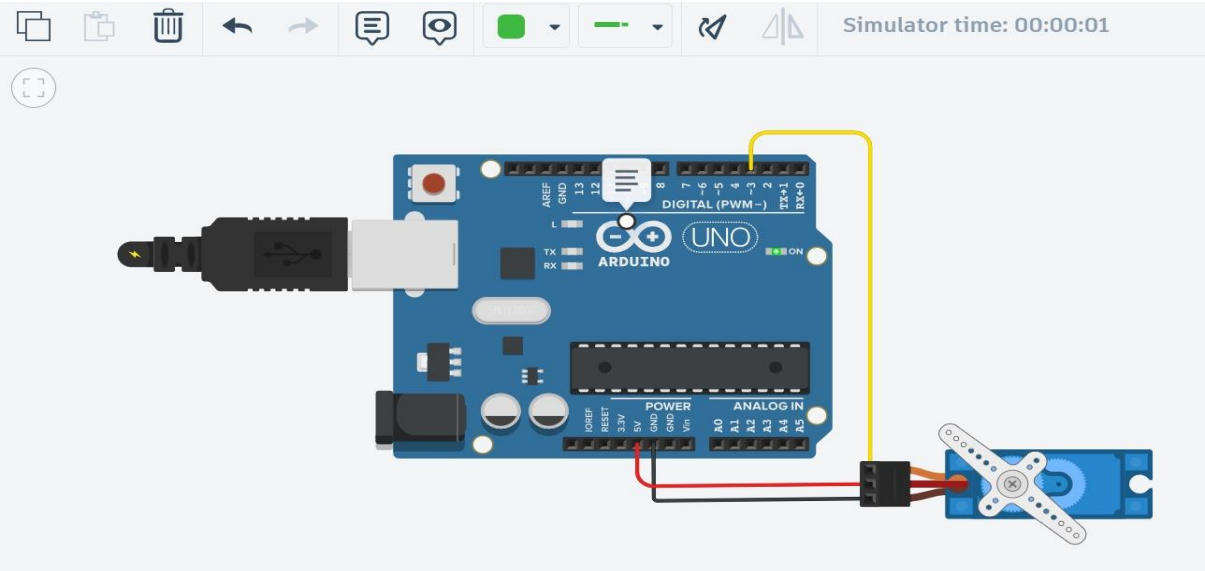


Fig. Servo Motor

Conclusion

This experiment successfully demonstrates servo motor control using Arduino:

- The servo motor was interfaced correctly using power (5V, GND) and a signal pin (D3).
- Using the Servo library, the servo was commanded to move to specific angles.
- The motor performed a continuous 0° → 180° → 0° sweep, proving accurate position control through PWM-based servo signalling.

Experiment Name: Pushdown Switch

Apparatus / Requirements: Arduino Uno, Light Dependent Resistor (LDR), Resistors (Two 10 kΩ and one 1 kΩ), LED, Push button (tactile switch) and connecting wires

Software Used: Tinkercad

Theory: A push button is a digital input device. It can represent two states:

Not pressed → LOW (0) and Pressed → HIGH (1)

However, when a switch input pin is not connected properly, the pin may remain floating, producing random HIGH/LOW values. To avoid this, we use a pull-down resistor.

Working Principle

1) Pushdown (Pull-Down) Switch Principle

One terminal of the push button is connected to +5V. The other terminal goes to a digital input pin (here, D7). A 10 kΩ pull-down resistor connects that input pin to GND.

- Button not pressed: input pin is held at GND through 10kΩ → LOW
- Button pressed: input pin directly receives +5V → HIGH

2) LDR Sensing: LDR + 10kΩ resistor forms a voltage divider, producing an analog voltage that Arduino reads using analogRead() (0–1023). An LDR changes resistance with light:

- More light → lower resistance Less light → higher resistance

3) When the push button is pressed, the program uses the sensor reading and controls output intensity using analogWrite() (PWM duty cycle).

Code:

```
// C++ code
//
void setup()
{ pinMode(3, INPUT);
  Serial.begin(9600);
  pinMode(2, OUTPUT);
}

void loop()
{
 int x = digitalRead(3);
  Serial.print(x);
  if(x==1)
  {
    digitalWrite(2,HIGH);

  }
  else
  {
    digitalWrite(2,LOW);
  }

}
```

Result

The Serial Monitor prints the button state and sensor value.

When the push button is pressed, the output changes (LED intensity changes according to the read value / condition in code).

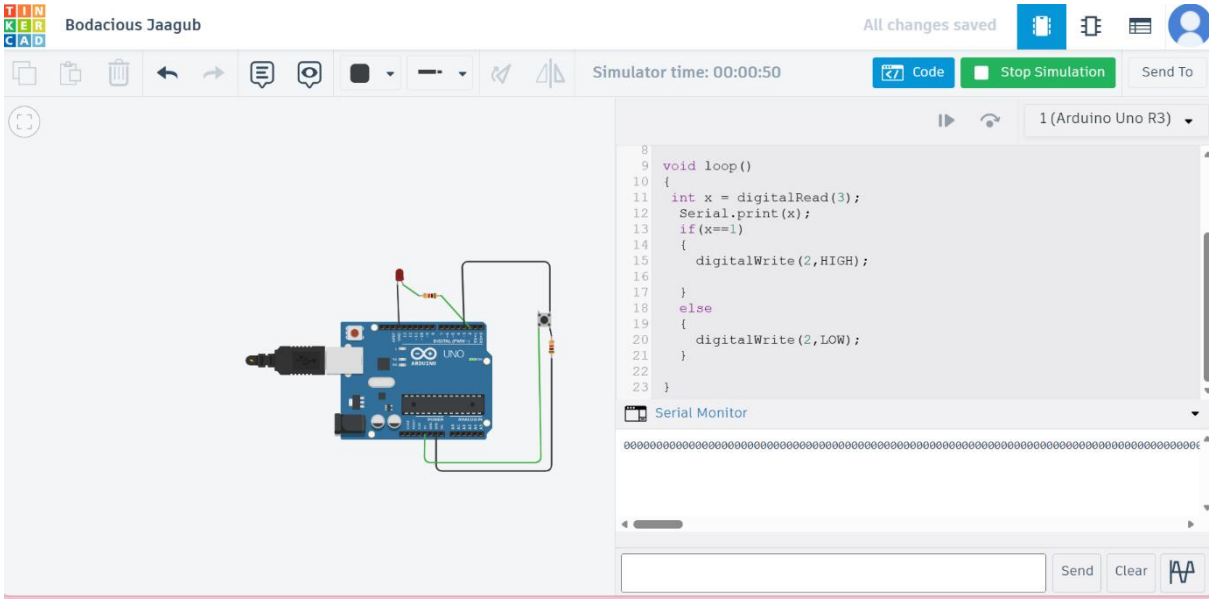


Fig. Pushdown Switch (Push down button unpressed)

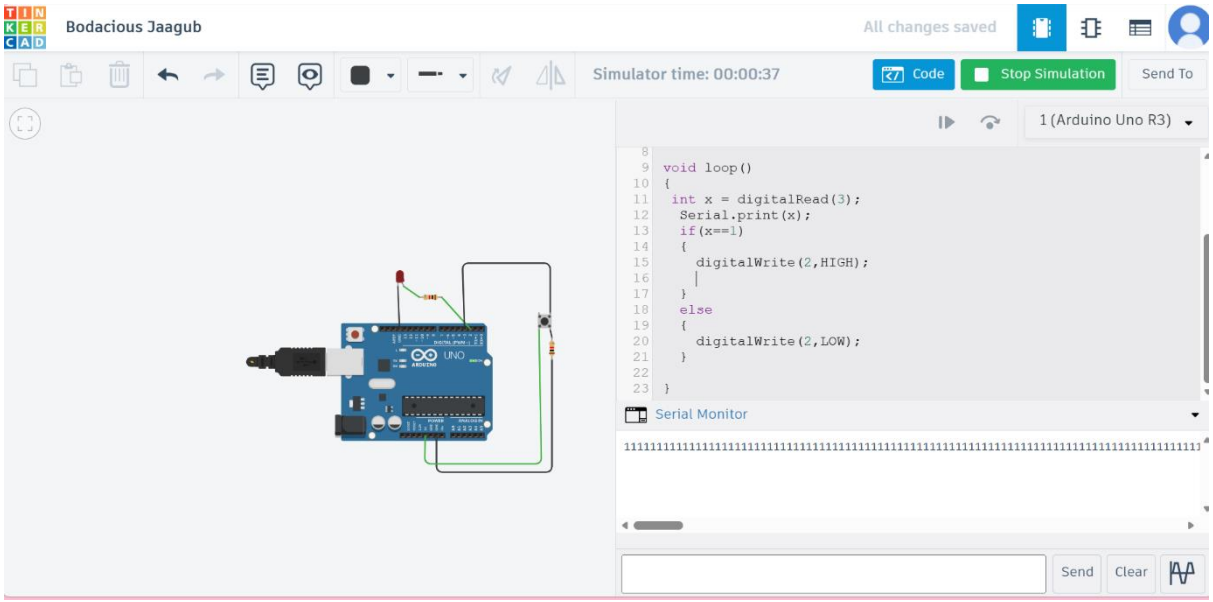


Fig. Pushdown Switch (Push down button pressed)

Conclusion

This experiment successfully demonstrates a pushdown (pull-down) switch using Arduino:

- The pull-down resistor ensures the input pin stays at a stable LOW when the button is not pressed (no floating input).
- On pressing the button, the input becomes HIGH, and the Arduino executes the control logic.
- With the LDR included, the circuit also demonstrates how a sensor input can be used conditionally (only when the button is pressed) to control an output (like LED brightness).

Experiment Name: LDR Interfacing using Arduino

Apparatus / Requirements: Arduino Uno, Light Dependent Resistor (LDR), Resistor (10 kΩ and 1 kΩ), LED, connecting wires

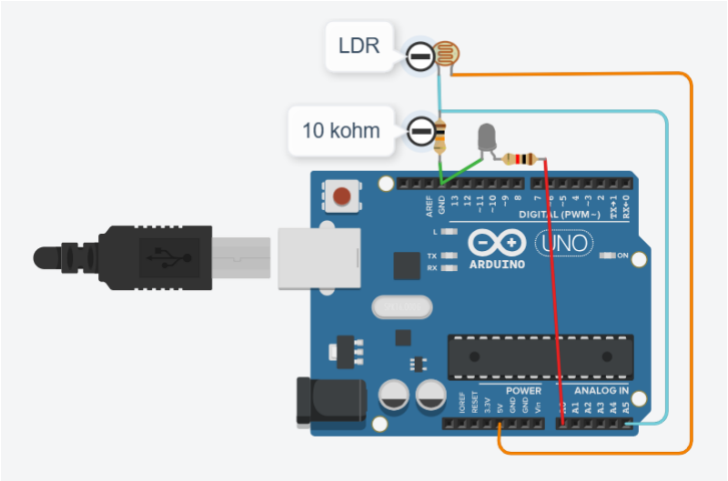
Hardware / Software Used: Tinkercad

Theory: An LDR (Light Dependent Resistor) is a passive electronic component whose resistance varies with the intensity of incident light. When light falls on the surface of the LDR, photons transfer energy to the electrons in the semiconductor material (commonly cadmium sulfide). This energy frees electrons, increasing conductivity and reducing resistance.

- High light intensity → Low LDR resistance
- Low light intensity → High LDR resistance

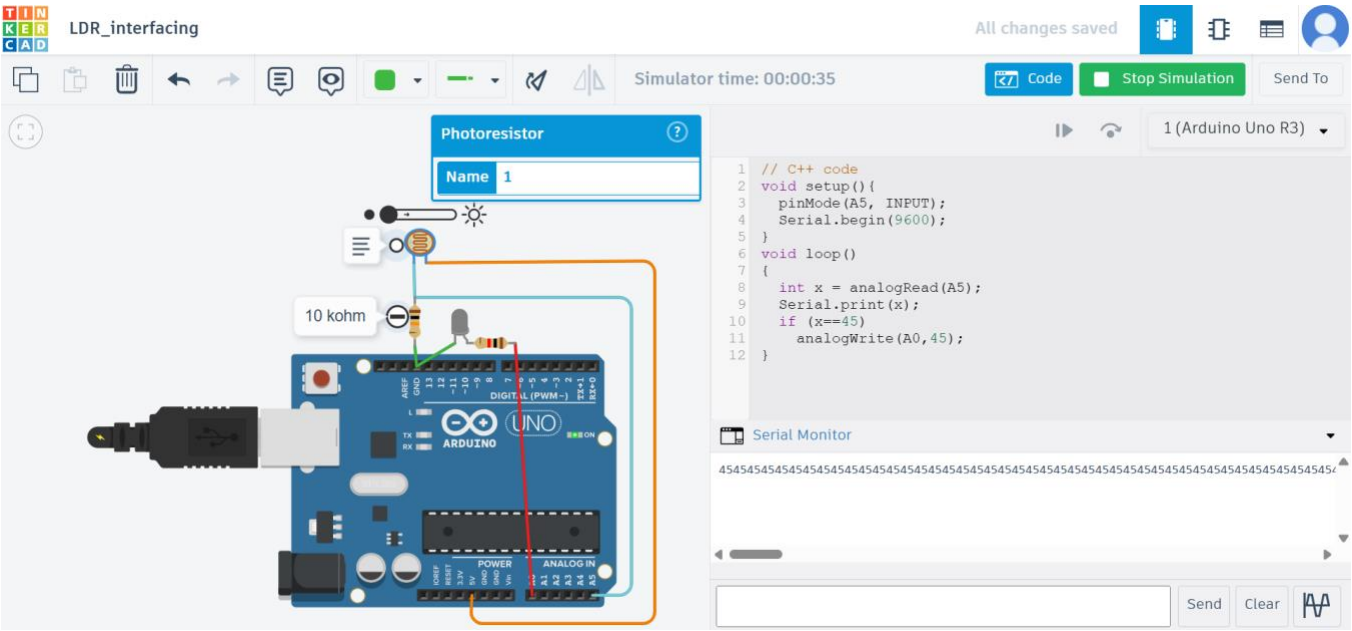
The LDR is connected in series with a fixed resistor to form a voltage divider. The output voltage of this divider changes with light intensity and is fed to an analog input pin of the Arduino.

The Arduino's ADC converts this analog voltage (0–5 V) into a digital value (0–1023). The microcontroller processes this value and executes predefined logic, such as activating an output device when light falls below or above a certain threshold.

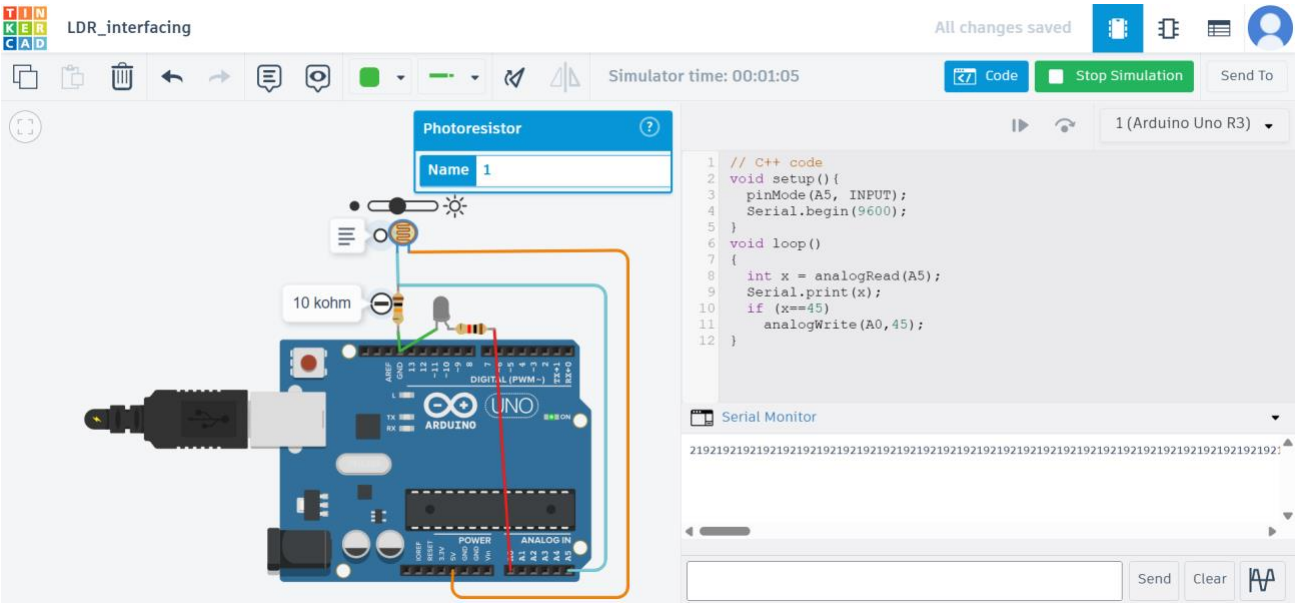



```
// C++ code
void setup(){
  pinMode(A5, INPUT);
  Serial.begin(9600);
}
void loop()
{
  int x = analogRead(A5);
  Serial.print(x);
  if (x==45)
    analogWrite(A0,45);
}
```

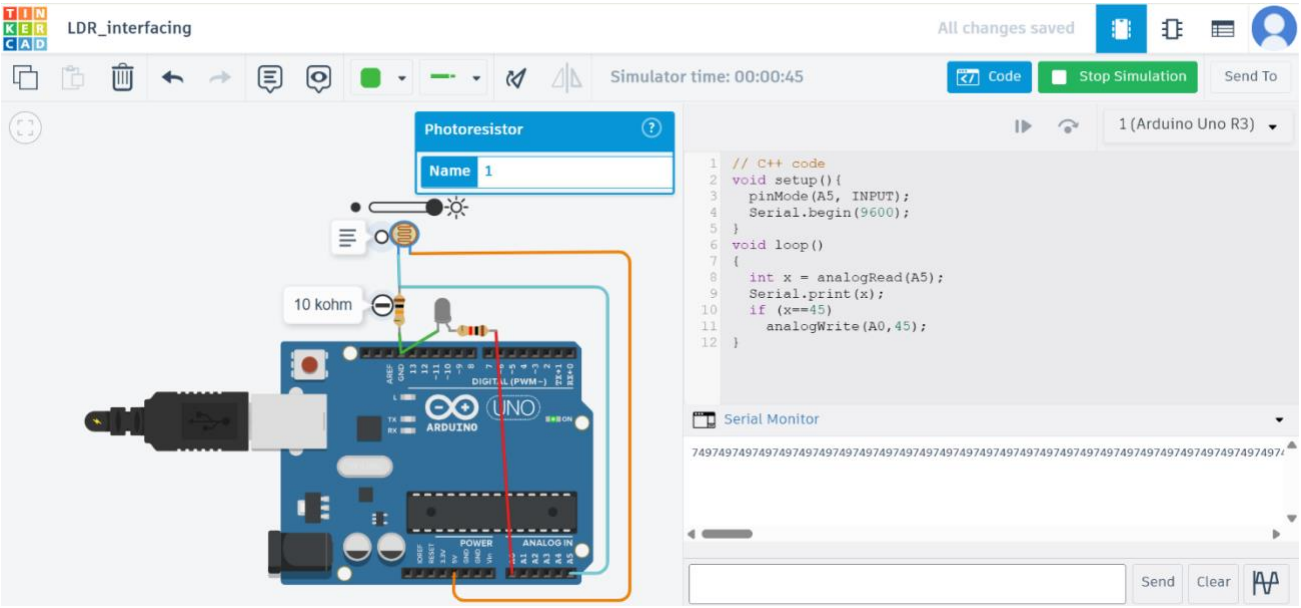
Result: When slider is at leftmost, 45 is the minimum analog value detected at lowest light level



When slider is at middle, 219 analog value detected at moderate light intensity.



When slider is at the rightmost, 293analog value indicates saturation or limited sensitivity of the LDR voltage divider at higher light levels.



Conclusion

LDR interfacing using Arduino demonstrates a simple and effective method of light sensing and control. By converting changes in light intensity into electrical signals, the Arduino can monitor environmental lighting conditions and automate responses accordingly. This experiment forms the basis for applications such as automatic street lights, light-controlled alarms, brightness control systems, and smart home automation.

ASSIGNMENT 1

Experiment 1 Name: Interfacing LED with Arduino UNO using Digital Pin.

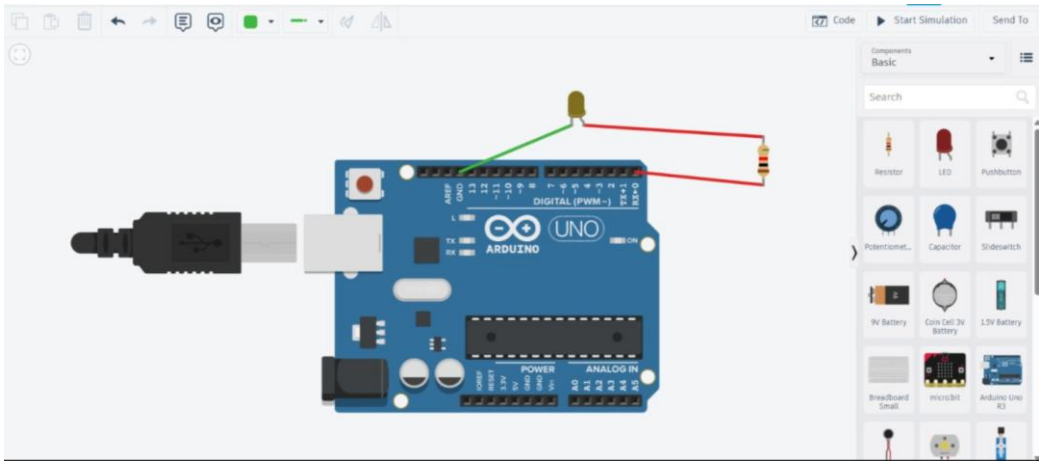
Apparatus / Requirements: Arduino UNO, LED, 220Ω Resistor, Connecting Wires, USB Cable, Computer/Laptop.

Hardware: Arduino UNO
Software: Tinkercad

Theory: Digital pins of Arduino UNO (0–13) are used for input and output operations. When configured as OUTPUT, they provide HIGH (5V) or LOW (0V) signals to control external devices such as LEDs.

Working Principle: When the digital pin is set to HIGH, current flows through the resistor and LED, causing it to glow. When set to LOW, the LED turns OFF. The resistor limits excess current.

RESULTS:



1 (Arduino Uno R3)

```
// C++ code
//
void setup()
{
  pinMode(0, OUTPUT);
}

void loop()
{
  digitalWrite(0 , HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(0 , LOW);
  delay(1000); // Wait for 1000 millisecond(s)
}
```

Conclusion: The experiment demonstrates successful LED interfacing using a digital pin of the Arduino UNO.

Experiment 2 Name: Interfacing LED with Arduino UNO using Analog Pin.

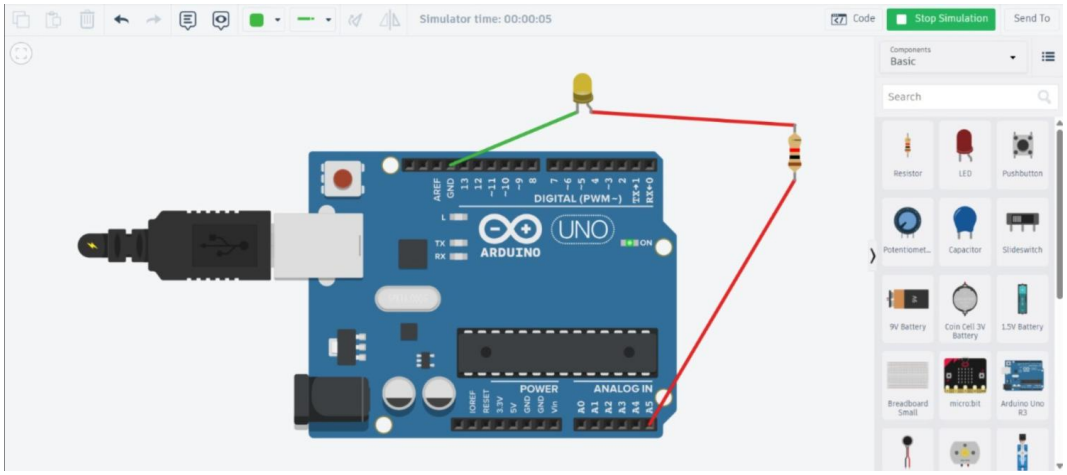
Apparatus / Requirements: Arduino UNO, LED, 220Ω Resistor, Connecting Wires, USB Cable, Computer/Laptop.

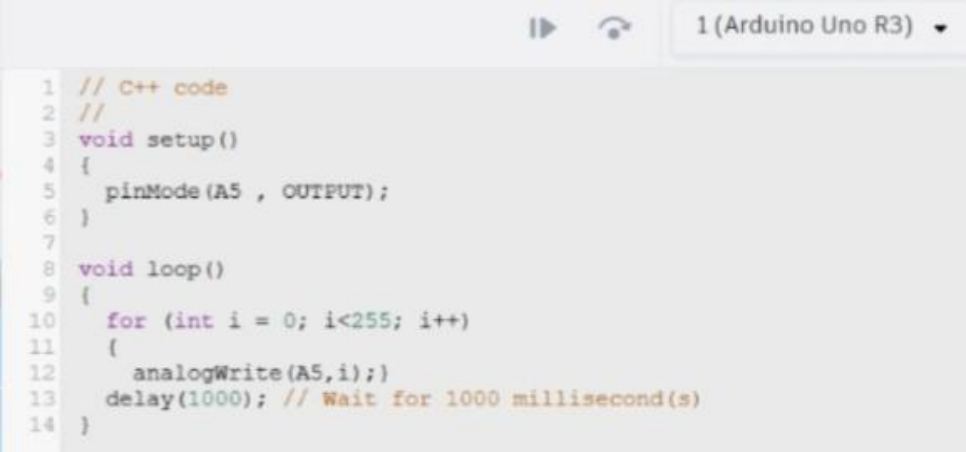
Hardware: Arduino UNO
Software: Tinkercad

Theory: Analog pins of Arduino UNO (A0–A5) are primarily used for analog input. However, they can also function as digital input/output pins. When configured as OUTPUT, they provide HIGH (5V) or LOW (0V) signals to control external devices such as LEDs.

Working Principle: When the analog pin (A5) is set as OUTPUT and written HIGH, current flows through the resistor and LED, causing it to glow. When set to LOW, the LED turns OFF. The resistor limits excess current.

RESULTS:





Conclusion: The experiment verifies that Arduino analog pins can also function as digital output pins for controlling an LED.